PROJECT REPORT

SPECTRAL VOICE INTELLIGENCE

AI-Based Speech Enhancement Using a Residual Attention U-Net

by

MOHAMED IMTHIYAS M

Academic Year: 2026–2027

DECLARATION

I hereby declare that the project report entitled “Spectral Voice Intelligence: AI-Based Speech Enhancement Using a Residual Attention U-Net” is prepared based on the project implementation, the project repository and the technical explanation supplied for this report. The work described is presented to the best of my knowledge. External reference material is acknowledged in the references section.

Place: Coimbatore

Signature: _____

Date: 27.09.2026

Name: Mohamed Imthiyas M

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, department faculty members and institution for their guidance and support. I also acknowledge the developers and maintainers of PyTorch and TorchAudio for their open-source tools and technical documentation, particularly the Audio Feature Extractions tutorial used as a reference for understanding waveform and time-frequency representations.

ABSTRACT

Spectral Voice Intelligence (SVI) is an AI-based speech enhancement project designed to reduce unwanted background noise from speech recordings while retaining speech-related information. The system converts an input recording into a time-frequency representation using the Short-Time Fourier Transform (STFT). The magnitude is transformed into a logarithmic scale and normalized before being supplied to a Residual Attention U-Net. The network learns from paired noisy and clean speech recordings and predicts a clean magnitude spectrogram. During reconstruction, the predicted magnitude is combined with the phase information from the noisy input and converted back into a waveform using the Inverse STFT (ISTFT).

The training implementation uses a supervised learning approach, a combined L1 and Mean Squared Error (MSE) loss, the AdamW optimizer, validation monitoring, learning-rate scheduling and early stopping. The repository configuration specifies a 16 kHz sample rate, 4-second audio segments, FFT size 512, hop length 128, window length 512, batch size 1 and an initial learning rate of 0.0001. A training run described in the project discussion reported a best checkpoint at epoch 21 with validation loss 0.165630 and early stopping at epoch 31. The project discussion also recorded a dataset-count discrepancy that must be resolved before treating the result as a final quality benchmark.

The project demonstrates the integration of digital signal processing and deep learning for speech enhancement. Objective audio-quality measurements and controlled listening tests are required for a complete evaluation.

KEYWORDS

Speech enhancement, noise reduction, STFT, spectrogram, Residual Attention U-Net, supervised learning, PyTorch, TorchAudio, ISTFT.

TABLE OF CONTENTS

- 1. Introduction
- 2. Problem Definition and Objectives
- 3. Existing and Proposed System
- 4. Tools and Technologies
- 5. System Design and Architecture
- 6. Methodology and Algorithms
- 7. Model Training Procedure
- 8. Inference and Audio Reconstruction
- 9. Implementation Details
- 10. Testing and Results
- 11. Limitations and Future Enhancements
- 12. Conclusion
- References
- Appendix A – Configuration Summary
- Appendix B – PyTorch Tutorial Image References

Note: In Microsoft Word, the table of contents can be updated automatically if heading styles are retained. This report uses Word heading styles for that purpose.

1. INTRODUCTION

1.1 Background

Speech recordings are frequently affected by environmental sounds such as fans, traffic, machinery and nearby conversations. These sounds can reduce intelligibility and make recordings less suitable for communication, transcription and other speech-based applications. Speech enhancement aims to improve the perceptual clarity of a speech signal by reducing unwanted components.

Traditional noise reduction may use fixed filters or statistical assumptions. Such approaches can be less effective when the background noise changes over time or overlaps with speech frequencies. Deep learning provides another approach: a neural network can learn patterns from examples of noisy speech and corresponding clean speech.

1.2 Project Overview

SVI processes audio in the time-frequency domain rather than passing raw waveform samples directly to the main neural network. The STFT produces a spectrogram that represents frequency content over time. The model estimates a cleaner magnitude representation, and the reconstruction stage converts the estimated representation back into audible waveform audio.

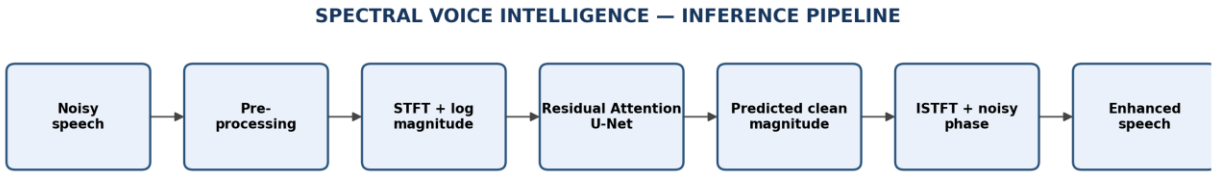


Figure 1.1. SVI inference pipeline. This is an original explanatory schematic prepared for this report; it is not a screenshot from the PyTorch tutorial.

1.3 Scope

- Preparing speech recordings in a consistent mono, sample-rate and duration format.
- Transforming audio into STFT-based magnitude and phase representations.
- Training a Residual Attention U-Net on paired noisy and clean speech.
- Predicting a cleaner magnitude spectrogram from a noisy input.
- Reconstructing enhanced waveform audio using the noisy phase and predicted magnitude.

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Definition

A speech recording may contain both the desired speaker and unwanted background noise. The project addresses the problem of estimating a cleaner speech representation from a noisy recording. The main technical challenge is to suppress noise without removing important speech details.

2.2 Objectives

- Standardize input recordings through channel conversion, resampling, trimming or padding, and amplitude normalization.
- Represent audio using STFT magnitude and phase information.
- Use log-magnitude transformation and normalization to prepare stable model inputs.
- Design and train a Residual Attention U-Net to predict clean spectrogram magnitudes.
- Use paired noisy-clean recordings for supervised learning.
- Optimize the network using a combined L1 and MSE loss with AdamW.
- Apply validation monitoring and early stopping to select a useful checkpoint.
- Reconstruct the predicted spectrogram into a playable audio waveform.

2.3 Expected Outcome

The intended output is a speech recording in which background noise is reduced and speech is clearer. The actual improvement depends on the paired dataset, training coverage, model behaviour and reconstruction quality. A lower validation loss alone does not prove that the enhanced audio is perceptually better.

3. EXISTING AND PROPOSED SYSTEM

3.1 Existing Approaches

Conventional audio processing may use fixed filters, spectral subtraction or other signal-processing methods. These methods can be useful for particular noise conditions, but their performance may be limited when noise characteristics vary or overlap with speech.

3.2 Proposed System

The proposed system uses a supervised deep-learning model operating on a log-magnitude spectrogram. A Residual Attention U-Net extracts multi-scale features, uses residual blocks to support information flow and applies attention gates to reweight feature information. The model predicts a clean magnitude spectrogram that is reconstructed with the noisy input phase.

3.3 Advantages

- Learns a mapping from noisy speech to clean speech using paired examples.
- Uses a time-frequency representation that exposes local spectral patterns.
- Combines encoder-decoder feature extraction, residual learning and attention.
- Supports checkpoint saving and inference on CPU or CUDA-compatible GPU.
- Separates preprocessing, model, training and inference into project modules.

4. TOOLS AND TECHNOLOGIES

Technology	Role in the project
Python	Main programming language for data processing, model training and inference.
PyTorch	Tensor operations, neural-network layers, automatic differentiation and optimization.
TorchAudio	Audio loading, resampling and signal transforms such as STFT/ISTFT.
NumPy	Numerical data support where required by the audio pipeline.
SoundFile / Librosa	Audio file handling and supporting audio utilities listed in the project dependencies.

Matplotlib	Visualisation of audio and training-related information when required.
Git and GitHub	Version control and source-code hosting.

4.1 Hardware and Software Requirements

Category	Requirement / note
Python	Python 3.10 or newer as stated in the repository README.
Training device	CUDA-capable NVIDIA GPU is recommended; CPU is possible but slower.
Memory	Repository recommends at least 8 GB system RAM for training.
GPU memory	Repository recommends 4 GB or more VRAM as a starting point.
Operating system	Windows, Linux or macOS, subject to compatible PyTorch/TorchAudio installation.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Overall Architecture

The system has two related workflows: training and inference. During training, noisy and clean recordings are paired and transformed into model-ready spectrograms. The model prediction is compared with the clean target, and its parameters are updated. During inference, a single noisy recording is transformed and passed through the trained model; the predicted magnitude is reconstructed into waveform audio.

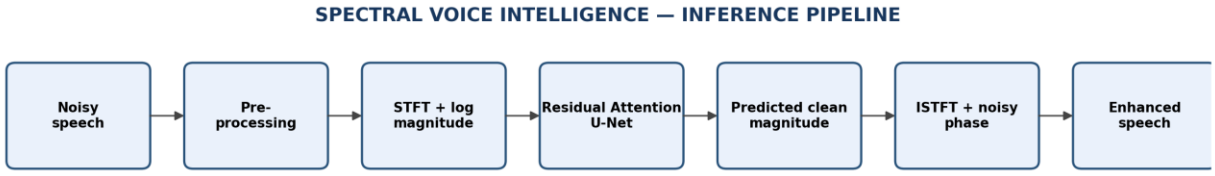


Figure 5.1. End-to-end inference flow for SVI.

5.2 Residual Attention U-Net

The U-Net structure contains an encoder and a decoder. The encoder progressively extracts features at different resolutions. The bottleneck processes deeper feature representations. The decoder reconstructs an output representation. Skip connections transfer information from encoder stages to corresponding decoder stages so that useful spatial and frequency-time detail can be retained.

Residual blocks use shortcut connections. A simplified residual operation is Y = F(X) + X, where X is the incoming feature map and F(X) is the transformation learned by a group of layers. Attention gates reweight features so that informative regions can receive greater emphasis. These mechanisms do not guarantee perfect speech/noise separation; the network learns their behaviour from the training data.

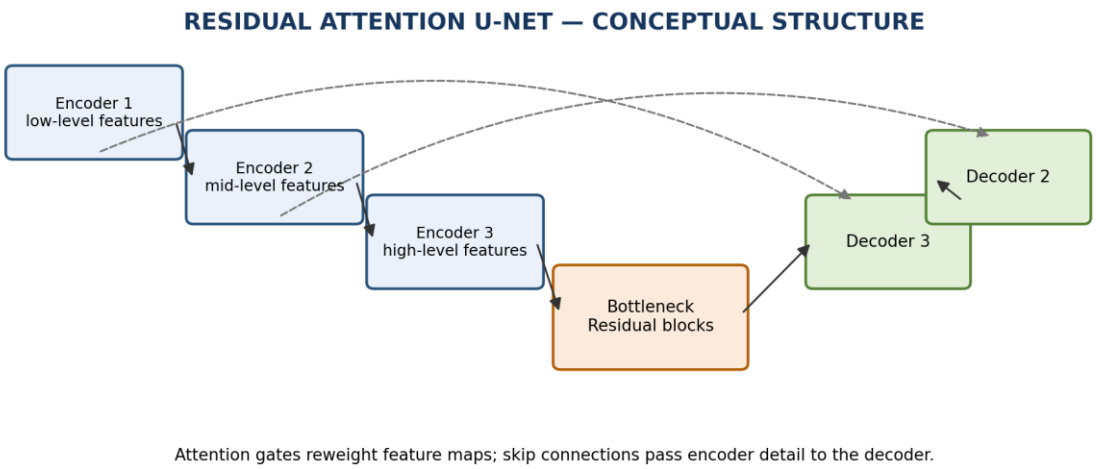


Figure 5.2. Conceptual Residual Attention U-Net layout. The diagram is explanatory and does not claim to reproduce every layer or tensor dimension in the source code.

5.3 Main Project Modules

Module / file	Responsibility
config.py	Central configuration: paths, sample rate, audio duration, STFT parameters, training settings and device.
preprocessing.py	Prepares waveform data for consistent model input.
dataset_loader.py	Pairs noisy and clean recordings using matching file names.
models/unet.py	Defines the Residual Attention U-Net model.
models/blocks.py	Contains reusable network blocks.
models/attention.py	Implements attention components.
models/loss.py	Contains model loss functionality where applicable.
train.py	Loads paired data, trains the model, validates and saves checkpoints.
inference.py	Loads a trained checkpoint and enhances an input recording.
test_*.py	Checks dataset, model blocks, attention, loss and tensor shapes.

6. METHODOLOGY AND ALGORITHMS

6.1 Audio Preprocessing

Audio preprocessing standardizes recordings before feature extraction. The project discussion identifies mono conversion, resampling to 16,000 Hz, padding, trimming and amplitude normalization. The current repository configuration specifies an audio duration of 4 seconds, giving a target length of 64,000 samples at 16 kHz.

Operation	Description	Purpose
Mono conversion	Converts multi-channel audio to one channel.	Keeps the model input channel format consistent.
Resampling	Converts audio to 16,000 samples per second.	Standardizes the sampling rate.
Trimming	Cuts recordings longer than the configured duration.	Controls input length.
Padding	Adds zeros when a recording is shorter than the configured duration.	Produces a fixed-length segment.
Waveform normalization	Scales waveform values as implemented in preprocessing.	Keeps numerical magnitudes manageable.

6.2 Short-Time Fourier Transform (STFT)

The Short-Time Fourier Transform converts a waveform from the time domain into a time-frequency representation. It analyses short overlapping frames of the signal. Each frame is transformed into frequency components, producing a spectrogram.

For a discrete signal x[n], a simplified STFT expression is:

$$X(m, k) = \Sigma_n \, x[n] \, w[n - mH] \, e^{(-j2\pi kn/N)}$$

Here, w is the analysis window, H is the hop length, N is the FFT size, m identifies the time frame and k identifies the frequency bin.

Parameter	SVI configuration	Meaning
Sample rate	16,000 Hz	Number of samples per second.
FFT size (N_FFT)	512	Controls the number of FFT points and frequency-bin resolution.
Window length	512	Number of samples in each analysis window.
Hop length	128	Frame advance; adjacent frames overlap.

With a 512-point FFT, the one-sided real-valued spectrum has 257 frequency bins. The hop length of 128 means that each new frame advances by 128 samples.

6.3 Magnitude and Phase Extraction

The STFT is complex-valued. It contains magnitude and phase. Magnitude indicates the strength of each time-frequency component, while phase carries angle information needed for waveform reconstruction. In the described SVI inference pipeline, the neural network predicts a cleaner magnitude representation and the noisy input phase is reused during reconstruction.

6.4 Log-Magnitude Spectrogram

Spectral magnitudes can span a wide numerical range. A logarithmic transformation compresses this range and makes weaker and stronger components easier to represent together.

$$L = \log(M + \varepsilon)$$

M represents magnitude and ε is a small positive constant that prevents taking the logarithm of zero. The project configuration defines EPSILON = 1e−7.

6.5 Spectrogram Normalization

The project discussion describes standardization using the mean and standard deviation:

$$X_norm = (X - \mu) / (\sigma + \varepsilon)$$

X is the original log-spectrogram value, μ is the mean and σ is the standard deviation. The noisy input statistics are also used to normalize the corresponding clean target in the described training method. This makes input and target scales consistent for supervised learning.

6.6 Residual Attention U-Net

The model receives a normalized noisy log-magnitude spectrogram and predicts a normalized clean spectrogram. Its main parts are:

- Encoder: extracts local and progressively higher-level time-frequency features.
- Residual blocks: add shortcut paths to support feature and gradient flow.
- Attention gates: learn feature weights that emphasize informative representations.
- Bottleneck: processes compressed high-level information.
- Decoder: reconstructs the predicted spectrogram representation.
- Skip connections: pass corresponding encoder features to decoder stages.

6.7 Supervised Learning with Paired Audio

Supervised learning trains a model using input examples and corresponding target outputs. SVI uses matching noisy and clean audio files. A noisy recording is the input, and its clean counterpart is the target. File names must match so that the correct pair is loaded.

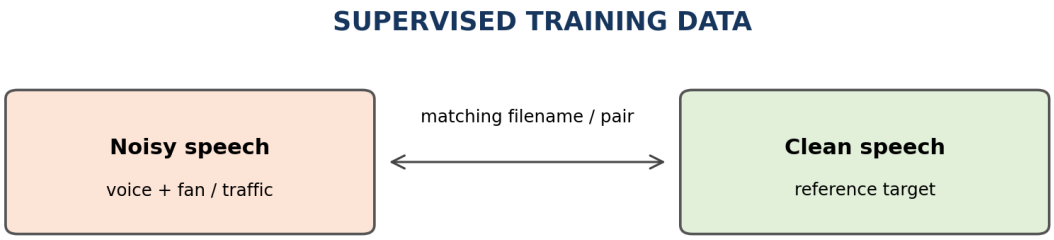


Figure 6.1. Paired noisy-clean data used for supervised learning.

The repository README gives the dataset structure as dataset/clean_speech/ and dataset/noisy_speech/. The dataset itself is not included in the repository and must be supplied separately.

6.8 Combined L1 and MSE Loss

The training implementation combines L1 loss and Mean Squared Error (MSE). L1 measures absolute differences. MSE squares the differences and therefore penalizes larger errors more strongly.

$$\text{Total Loss} = 0.8 \times L1 + 0.2 \times MSE$$

The implementation uses 80% weight for L1 and 20% for MSE. This combination makes absolute error the main objective while retaining a penalty for larger deviations.

6.9 AdamW Optimizer and Learning-Rate Scheduler

AdamW updates the learnable model parameters using gradients and adaptive learning rates. The source configuration specifies an initial learning rate of 0.0001 and weight decay of 0.00001. The training script also uses a ReduceLROnPlateau scheduler: when validation loss stops improving, the learning rate is reduced by a factor of 0.5, subject to a minimum learning rate of 0.000001.

6.10 Gradient Clipping and Early Stopping

The training script clips the gradient norm to 1.0 to limit excessively large gradient updates. Early stopping monitors validation loss. The configured patience is 10 epochs: if validation loss does not improve for ten consecutive epochs, training is stopped. The best validation checkpoint is saved separately from the latest checkpoint.

6.11 Inverse STFT (ISTFT)

The Inverse STFT converts the estimated complex time-frequency representation back into a waveform. SVI combines the predicted magnitude with the noisy input phase, forms a complex spectrogram and performs inverse transformation. The result is saved as an enhanced audio file. Since the noisy phase is reused, the output is not a fully phase-estimated reconstruction.

6.12 Methods from the PyTorch Audio Feature Tutorial

The supplied PyTorch/TorchAudio tutorial is used as a visual and conceptual reference for waveform plotting, spectrograms, FFT-size comparisons, resampling, Griffin-Lim reconstruction, Mel filter banks, Mel spectrograms, MFCC, LFCC and pitch. These are tutorial demonstrations and must not be confused with the core SVI model pipeline. SVI's documented model path is based on STFT log magnitude, a Residual Attention U-Net and ISTFT.

Tutorial visual / feature	How it relates to SVI
Waveform and spectrogram	Directly relevant to understanding the input waveform and STFT representation.
FFT-size comparison	Explains the effect of FFT size; SVI uses N_FFT = 512.
Resampling comparison	Supports the need for a consistent sample rate; SVI uses 16 kHz.
Griffin-Lim reconstruction	A tutorial reconstruction method; SVI's described pipeline uses noisy phase with predicted magnitude and ISTFT.
Mel filter bank / Mel spectrogram	Additional audio feature representations; not specified as the main SVI model input.
MFCC and LFCC	Alternative compact feature representations; not specified as the core SVI model input.
Pitch feature	An additional speech-analysis feature; not part of the stated core enhancement pipeline.

7. MODEL TRAINING PROCEDURE

7.1 Dataset Preparation

1. Place clean recordings in dataset/clean_speech/.
2. Place their noisy counterparts in dataset/noisy_speech/.
3. Use identical filenames for each corresponding pair.
4. Run the dataset test to verify that pairs are detected.
5. Check that audio files can be loaded and converted to the expected tensor shape.

7.2 Training and Validation Split

The training code uses a fixed random seed of 42 and reserves 20% of the paired dataset for validation. The remaining 80% is used for training. The validation set is used to monitor generalization and select the best checkpoint.

7.3 Training Steps

6. Load a batch of noisy and clean waveforms.
7. Move tensors to the selected CPU or CUDA device.
8. Convert waveforms to log-magnitude spectrogram representations and normalize them.
9. Pass noisy spectrograms through the Residual Attention U-Net.
10. Compare predicted spectrograms with clean target spectrograms using the combined loss.
11. Backpropagate the loss and clip gradient norms.

12. Update model parameters using AdamW.
13. Calculate validation loss after each epoch.
14. Update the learning-rate scheduler and save the latest checkpoint.
15. Save the best checkpoint whenever validation loss improves; stop when patience is reached.

7.4 Configuration Summary

Setting	Configured value
Sample rate	16,000 Hz
Audio duration	4 seconds (current config.py)
Maximum waveform length	64,000 samples
N_FFT	512
Hop length	128
Window length	512
Batch size	1
Maximum epochs	100
Learning rate	0.0001
Weight decay	0.00001
Base channels	16
Bottleneck residual blocks	2
Validation ratio	20%
Gradient clipping norm	1.0
Early-stopping patience	10
Random seed	42

Configuration note: the repository README contains an older quick-start summary that mentions a 2-second segment. The current config.py source specifies AUDIO_DURATION = 4. This report follows the current config.py value.

8. INFERENCE AND AUDIO RECONSTRUCTION

8.1 Inference Procedure

16. Load the trained best checkpoint.
17. Read the selected input audio file.
18. Convert the waveform to mono and resample to 16 kHz where required.
19. Prepare the waveform to the configured duration.
20. Calculate STFT and separate magnitude and phase.
21. Apply log transformation and normalization to the magnitude.
22. Run the trained Residual Attention U-Net to predict clean magnitude.
23. Reverse normalization and logarithmic scaling as implemented by the inference pipeline.
24. Combine the predicted magnitude with the noisy input phase.
25. Apply ISTFT and save the output waveform in the outputs directory.

8.2 Output

The repository describes enhanced audio output being written to the outputs/ directory, for example outputs/enhanced_audio.wav. The exact supported file formats depend on the audio conversion implementation and installed audio libraries.

8.3 Reconstruction Consideration

Because the reconstruction uses the noisy recording's phase, residual noise or phase-related artefacts may remain. A future version could investigate phase-aware prediction, complex-valued enhancement or alternative reconstruction approaches. Such changes should be evaluated rather than assumed to improve results.

9. IMPLEMENTATION DETAILS

9.1 Repository Structure

```
Spectral-Voice-Intelligence/
├── models/
│   ├── unet.py
│   ├── attention.py
│   ├── blocks.py
│   └── loss.py
├── dataset/
│   ├── clean_speech/
│   └── noisy_speech/
├── checkpoints/
├── outputs/
├── temp/
├── config.py
├── preprocessing.py
├── dataset_loader.py
├── train.py
├── inference.py
├── audio_converter.py
├── test_attention.py
├── test_blocks.py
├── test_dataset.py
├── test_loss.py
├── test_shape.py
├── test_unet.py
└── requirements.txt
```

9.2 Running the Project

The repository README documents the following general workflow. Commands may need minor adjustment based on the local folder name and Python environment.

```
git clone https://github.com/MohamedImthiyas-M/Spectral-Voice-Intelligence.git

cd Spectral-Voice-Intelligence

python -m venv .venv

.venv\Scripts\activate   (Windows)

python -m pip install --upgrade pip

pip install -r requirements.txt

python test_dataset.py

python test_unet.py

python test_shape.py

python train.py
```

```
python inference.py
```

9.3 Checkpoint and Output Folders

The training script saves the best model and the latest model under checkpoints/. The inference stage writes enhanced audio to outputs/. These folders are created by the project configuration when the program starts.

10. TESTING AND RESULTS

10.1 Functional Tests

Test	Purpose	Expected / reported behaviour
Dataset test	Check matching noisy-clean filenames.	Valid paired files should be detected.
U-Net test	Check model forward pass and output shape.	Model should process spectrogram tensors without shape errors.
Shape test	Check noisy, clean and phase tensor dimensions.	Frequency dimension is consistent with the 512-point FFT (257 one-sided bins).
Attention test	Check attention module operation.	Attention module should produce valid feature outputs.
Blocks test	Check reusable model blocks.	Blocks should accept and return expected tensor shapes.
Loss test	Check combined loss calculation.	Loss should return a finite scalar for valid tensors.

10.2 Training Result Reported in Project Discussion

Metric	Reported value
Best checkpoint epoch	21
Best validation loss	0.165630
Training stopped at epoch	31
Configured maximum epochs	100
Early-stopping patience	10

These values are taken from the project explanation supplied for this report. The raw training log was not included as a separate file, so the values are recorded as project-reported results rather than independently verified measurements.

10.3 Dataset Count Discrepancy

The project discussion notes that the dataset loader detected 20 pairs in a later run, whereas an earlier dataset was expected to contain approximately 1,217 pairs. The training script itself includes a safeguard that stops training when fewer than 10 pairs are found and warns that the earlier dataset contained about 1,217 pairs. The current detected count must therefore be verified before claiming that the model was trained on the full dataset.

Recommended verification: check that every clean file has a noisy counterpart with exactly the same filename, confirm the dataset folders are in the configured project root, and rerun the dataset test. Do not report 1,217 as the actual count unless the test output confirms it.

10.4 Evaluation Measures

The reported validation loss measures the model's spectrogram prediction error. It is not, by itself, an objective speech-quality score. A stronger evaluation should include paired audio comparisons and established metrics such as PESQ, STOI and SI-SDR where the dataset and evaluation protocol support them. Listening tests should also compare the original noisy signal and enhanced output at matched loudness.

10.5 Result Interpretation

The reported best checkpoint indicates that validation loss reached its lowest observed value at epoch 21 in the described run. Early stopping at epoch 31 is consistent with ten subsequent epochs without improvement. This result demonstrates that the training control logic operated as intended in that run, but it does not independently establish the amount of audible noise reduction.

11. LIMITATIONS AND FUTURE ENHANCEMENTS

11.1 Current Limitations

- The dataset is not bundled with the public repository; reproducibility depends on the dataset supplied by the user.
- The project discussion records a discrepancy between 20 detected pairs and an earlier expectation of approximately 1,217 pairs.
- The model predicts magnitude while reusing noisy phase during reconstruction.
- Validation loss alone does not prove perceptual speech quality.
- Performance may vary with speaker, microphone, noise type, signal-to-noise ratio and recording conditions.
- A complete set of objective speech-quality scores and controlled listening-test results was not supplied for this report.

11.2 Future Enhancements

- Restore and verify the intended full paired dataset, then document the exact train/validation sample counts.
- Evaluate enhanced audio with PESQ, STOI and SI-SDR where applicable, alongside listening tests.
- Investigate phase-aware or complex-spectrogram enhancement to reduce reconstruction artefacts.
- Test generalization on unseen speakers and noise conditions.
- Compare model performance with a simple baseline such as spectral subtraction or a conventional Wiener filter.
- Add reproducible experiment logs, configuration snapshots and plots of training and validation loss.
- Improve inference interface and provide clear input/output validation and error messages.

12. CONCLUSION

Spectral Voice Intelligence demonstrates a speech-enhancement workflow that combines digital signal processing with deep learning. Audio preprocessing standardizes the input, STFT converts the waveform into a time-frequency representation, and log-magnitude normalization prepares the spectrogram for learning. A Residual Attention U-Net is trained using paired noisy and clean speech with a weighted L1-MSE objective and AdamW optimization. Validation monitoring, learning-rate scheduling, gradient clipping

and early stopping support the training process. During inference, the predicted magnitude is combined with noisy phase and reconstructed using ISTFT.

The supplied project discussion reports a best validation loss of 0.165630 at epoch 21 and early stopping at epoch 31. The dataset count discrepancy remains an important issue to resolve, and formal audio-quality evaluation is needed before making strong claims about enhancement performance. Overall, the project provides a practical foundation for experimenting with AI-based speech enhancement.

REFERENCES

1. Mohamed Imthiyas M., “Spectral Voice Intelligence,” GitHub repository, main branch. https://github.com/MohamedImthiyas-M/Spectral-Voice-Intelligence
2. Moto Hira, “Audio Feature Extractions,” TorchAudio documentation. https://docs.pytorch.org/audio/stable/tutorials/audio_feature_extractions_tutorial.html
3. PyTorch Audio, “Audio Feature Extractions” Jupyter Notebook, source notebook referenced by the user. https://colab.research.google.com/github/pytorch/audio/blob/gh-pages/main/_downloads/743bf71677318e38b748ce39acd11a62/audio_feature_extractions_tutorial.ipynb
4. PyTorch, “TorchAudio: Audio and Speech Processing.” https://github.com/pytorch/audio
5. PyTorch documentation, “torch.optim.AdamW.” https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html

APPENDIX A – CONFIGURATION SUMMARY

The values below are taken from the current config.py source inspected in the public repository.

Parameter	Value
SAMPLE_RATE	16000
AUDIO_DURATION	4 seconds
N_FFT / HOP_LENGTH / WIN_LENGTH	512 / 128 / 512
EPSILON	1e-7
BATCH_SIZE / EPOCHS	1 / 100
LEARNING_RATE / WEIGHT_DECAY	1e-4 / 1e-5
GRAD_CLIP / PATIENCE	1.0 / 10
INPUT_CHANNELS / OUTPUT_CHANNELS	1 / 1
BASE_CHANNELS / BOTTLENECK_BLOCKS	16 / 2
DEVICE	CUDA when available; otherwise CPU

APPENDIX B – PYTORCH TUTORIAL IMAGE REFERENCES

The following are the exact official tutorial page and notebook references requested. The tutorial contains visual examples for waveform/spectrogram, FFT-size comparison, resampling, Griffin-Lim reconstruction, Mel filter bank, Mel spectrogram, MFCC, LFCC and pitch. The report's own figures are original explanatory schematics; they are not represented as screenshots of those tutorial outputs.

Official TorchAudio tutorial page: Audio Feature Extractions

Google Colab notebook: audio_feature_extractions_tutorial.ipynb

Official feature-overview image: torchaudio_feature_extractions.png

Tutorial section / figure	Use in report
Overview of audio features	Background reference for common TorchAudio transforms.
Original waveform and spectrogram	Illustrates time-domain waveform and time-frequency representation.
Effect of n_fft parameter	Illustrates frequency-resolution changes; SVI uses 512.
Resampling comparison	Illustrates why sample-rate consistency matters.
Griffin-Lim reconstruction	Reference for magnitude-based reconstruction; SVI uses noisy phase with ISTFT.
Mel filter bank and MelSpectrogram	Additional feature-extraction concepts, not the core SVI input representation.
MFCC and LFCC	Alternative audio feature representations, not specified as the SVI model input.
Pitch feature	Additional analysis feature, not part of the core SVI pipeline.

Image-use note: The official page and notebook links above point to the original tutorial material. This document includes original SVI-specific explanatory diagrams and links to the exact tutorial figures. The original rendered notebook plot images were not embedded as screenshots in this generated file.